

URL Shortener — Architecture Analysis

Project: URL Shortener
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic
Location: /home/dominic/Documents/fastapiurl
Report #: 2/20

Architecture Type

Monolithic REST API with SQLite persistence

Layers

API Layer → Service Layer → SQLAlchemy ORM → SQLite

Design Patterns

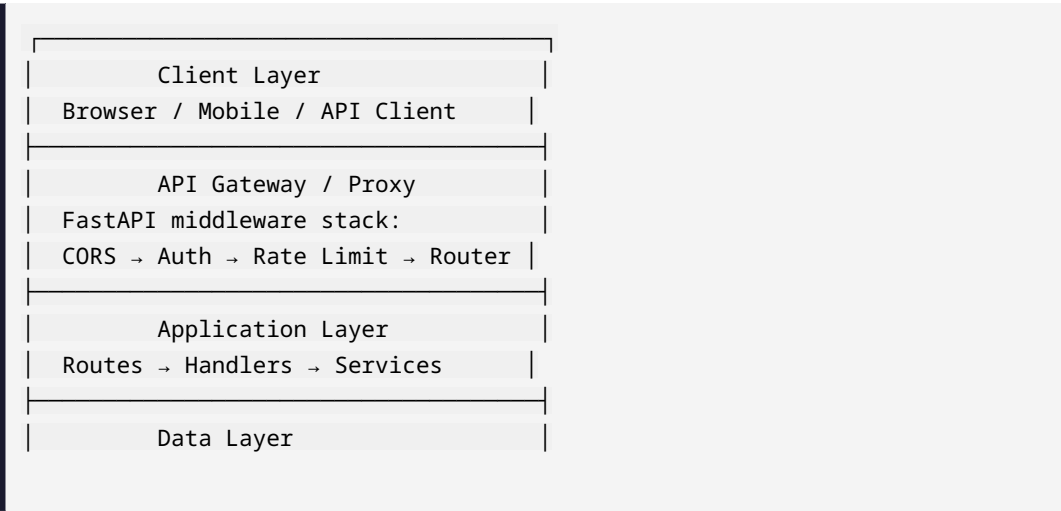
Router-Service pattern, Dependency Injection via FastAPI Depends

Scalability

Single-process, single-thread SQLite — suitable for low traffic

System Architecture

Layer Architecture



Component Interaction

URL Shortener Component Architecture

- └ API Layer (FastAPI)
- └ Service Layer (Business Logic)
- └ Data Layer (SQLAlchemy + SQLite)
- └ Client Layer (React 19)

Request Lifecycle

1. **HTTP Request** arrives at FastAPI
2. **CORS middleware** checks origin headers
3. **Auth dependency** extracts and validates JWT
4. **Rate limiter** checks request count per IP
5. **Router** matches path to handler
6. **Handler** executes business logic
7. **Database session** commits or rolls back
8. **Response** returned to client

Dependency Graph

URL Shortener Dependency Flow:

Client → FastAPI → Router → Handler → SQLite → Response

Design Patterns Used

Pattern	Location	Purpose
Dependency Injection	FastAPI Depends()	DB sessions, auth
Repository Pattern	SQLAlchemy models	Data abstraction
Middleware Chain	FastAPI middleware	Cross-cutting concerns
Router Module	APIRouter include	Separation of concerns